

Server Dokumentation – Angler

Der Spielstand-Server läuft als Node.js-Anwendung in einem Linux-Container (LXC CT 104) auf einem Proxmox-Heimserver. Von außen ist er über einen Cloudflare Tunnel unter **facescore.de** erreichbar.

Infrastruktur

Komponente	Details
Proxmox Host	Heimserver
Container	LXC CT 104 "facescore" – Ubuntu Linux
Node.js	v20.20.2
Prozessmanager	PM2
Port	3000 (intern), öffentlich über Cloudflare Tunnel

Dateistruktur

```
/var/www/angler/  
├─ server.js      - Haupt-Serverdatei mit allen API-Endpunkten  
├─ package.json   - Projektdefinition und Abhängigkeiten  
├─ node_modules/  - installierte Bibliotheken (express, cors)  
└─ data/          - Spielstände, je eine JSON-Datei pro Spieler  
    ├─ save_daniel.json  
    ├─ save_jannis.json  
    └─ ...
```

package.json

```
{  
  "name": "angler-server",  
  "version": "1.0.0",
```

```
"main": "server.js",  
"dependencies": {  
  "express": "^4.21.0",  
  "cors": "^2.8.5"  
}  
}
```

server.js

```
const express = require('express');  
const cors    = require('cors');  
const fs      = require('fs');  
const path    = require('path');  
  
const app = express();  
const PORT = 3000;  
const DATA = path.join(__dirname, 'data');  
  
app.use(cors());  
app.use(express.json());  
  
if (!fs.existsSync(DATA)) fs.mkdirSync(DATA, { recursive: true });  
  
// Sonderzeichen im Namen ersetzen damit der Dateiname sicher ist  
function savePath(name) {  
  return path.join(DATA, 'save_' + name.replace(/^[a-zA-Z0-9_-]/g, '_') + '.json');  
}  
  
app.post('/api/save', (req, res) => {  
  const body = req.body;  
  if (!body || !body.name) return res.status(400).json({ error: 'name required' });  
  try {  
    fs.writeFileSync(savePath(body.name), JSON.stringify(body, null, 2), 'utf8');  
    res.json({ ok: true });  
  } catch (e) {
```

```

        res.status(500).json({ error: e.message });
    }
});

app.get('/api/load/:name', (req, res) => {
    const p = savePath(req.params.name);
    if (!fs.existsSync(p)) return res.status(404).json({ error: 'no save found' });
    try {
        const data = JSON.parse(fs.readFileSync(p, 'utf8'));
        res.json(data);
    } catch (e) {
        res.status(500).json({ error: e.message });
    }
});

// wird vom Spiel beim Start aufgerufen um Internetverbindung zu prüfen
app.get('/api/status', (req, res) => {
    res.json({ ok: true, server: 'Angler Game Server', version: '1.0' });
});

app.listen(PORT, () => console.log('Angler Server running on port ' + PORT));

```

API-Endpunkte

Methode	Pfad	Beschreibung
POST	/api/save	Spielstand speichern (JSON-Body mit Spielerdaten)
GET	/api/load/:name	Spielstand laden – 404 wenn kein Eintrag gefunden
GET	/api/status	Prüfen ob Server erreichbar ist

Beispiel: gespeicherte Datei (data/save_daniel.json)

```
{
```

```
"name": "daniel",
"gold": 1500,
"rod": "Carbon Rod",
"bait": "Wurm",
"baitCount": 8,
"maxSlots": 14,
"unlockedZones": ["Startteich", "Ostsee"],
"inventory": [
  { "type": "Hecht", "weight": 2.3, "length": 41.0, "price": 120 }
],
"fishBook": {
  "Hecht": { "count": 5, "heaviest": 3.1, "longest": 52.0 }
}
}
```

PM2 – Server verwalten

```
pm2 start server.js --name angler
pm2 status
pm2 logs angler
pm2 restart angler
pm2 save
```

Proxmox – Container verwalten

```
pct list
pct status 104
pct enter 104

cd /var/www/angler
pm2 status
curl https://facescore.de/api/status
```

Kommunikation Spiel ↔ Server

Im Spiel übernimmt **SaveManager.java** die gesamte HTTP-Kommunikation. Verwendet wird `java.net.HttpURLConnection` – direkt in Java enthalten, keine externe Bibliothek nötig. JSON wird mit `org.json` (als JAR eingebunden) gebaut und geparkt. Speichern und Laden laufen auf eigenen Hintergrund-Threads damit das Spiel nicht einfriert.

Speichern:

`SaveManager.saveGame(player)`

- JSONObject aufbauen (Gold, Rute, Köder, Inventar, Fischbuch)
- Background-Thread starten
- POST `https://facescore.de/api/save`
- Server schreibt `data/save_<name>.json`

Laden:

GET `https://facescore.de/api/load/<name>`

- Server liest JSON-Datei
- `SaveManager.applyToPlayer()` stellt alle Felder wieder her

Kein Internet:

GET `/api/status` schlägt fehl → `serverReachable = false`

- Hinweis im Titelscreen und Pausemenü
- Spiellogik läuft weiter